

Multi-Class Network Intrusion Detection System

Cyber Threat Intelligence Project Report

April 2026

Introduction

Modern organization networks face a diverse and evolving range of cyber threats. The goal of a Cyber Threat Intelligence (CTI) team is to protect their company's assets and customers from cybersecurity risks. Real-world security operations require the ability to identify and classify multiple attack types simultaneously. A security analyst in a CTI team that is responding to a cyber threat needs to know that a breach is occurring and what type of attack it is in order to initiate an appropriate response.

This report presents a multi-class Network Intrusion Detection System (NIDS) trained to differentiate between twelve types of network traffic. One of the types of traffic is categorized as normal network traffic, while the rest are distinct network attack types. The dataset used does not utilize binary classification, which presents two key challenges: severe class imbalance and inter-class similarity. The severe class imbalance results from normal traffic significantly outnumbering attack traffic, and the inter-class similarity is due to certain attack types sharing similar network features.

Three machine learning models are trained and compared: Random Forest, Logistic Regression, and Decision Tree. Class weighting is applied to ensure that rare attack types have the appropriate attention during training. The results from the models are presented with macro-averaged metrics to provide a genuine assessment of per-class performance rather than aggregate accuracy inflated by the dominant normal traffic class.

Data

Dataset Description

The [CIC-IDS-2017](#) dataset is split into eight CSV files that captured network traffic across a five-day simulated environment. Each file represents a separate day and attack scenario that covers normal browsing activity alongside a range of distinctly different attack types. All files were used and concatenated into a single dataset for unified preprocessing and modeling.

There are three classes with fewer than 40 samples: Infiltration (36), Web Attack - SQL Injection (21), and Heartbleed (11). These classes were removed before modeling because they are sparse for any classifier to learn reliable patterns from. Additionally, their inclusion would bring in noise without adding predictive value. The altered dataset used is presented below:

Attribute	Value
Total flows	2,830,675
Raw features	78
Features after selection	47
Classes	12
Training samples (80%)	2,264,540
Test samples (20%)	566,135

Class Distribution

One challenge of the dataset is its severe class imbalance. BENIGN traffic (normal traffic) is 80.3% of all the network traffic flows, while several attack classes are less than 0.1% of the data. The table below shows the full class distribution:

Label	Count	Proportion
BENIGN	2,273,097	80.30%
DoS Hulk	231,073	8.16%
PortScan	158,930	5.61%
DDoS	128,027	4.52%
DoS GoldenEye	10,293	0.36%
FTP-Patator	7,938	0.28%
SSH-Patator	5,897	0.21%
DoS slowloris	5,796	0.20%
DoS Slowhttptest	5,499	0.19%
Bot	1,966	0.07%
Web Attack - Brute Force	1,507	0.05%
Web Attack - XSS	652	0.02%

Data Cleaning

An encoding artifact was identified in three Web Attack label names, producing garbled characters from a Latin-1/UTF-8 mismatch during file loading. These were corrected via direct string replacement. Following this fix, no missing values or infinite values were present in the dataset, requiring no further imputation.

Methodology

Feature Selection

Pearson correlation analysis was utilized across all 78 numeric features to remove the features that have an absolute correlation exceeding 0.90. One feature from each correlated pair was removed. This reduced the 78 features to 47 features. The reason for removing the highly correlated features is to avoid multicollinearity in the Logistic Regression model. If these features were not removed, the model would become unstable by struggling to assign stable coefficients to each feature. With Random Forest and Decision Tree, removing the correlated features improves the importance scores accuracy and interpretability.

Class Imbalance Handling

Due to the severe class imbalance between BENIGN and the attack types, training without introducing class weights would produce a model that is biased toward predicting BENIGN regardless of the network's true class. Class weights were computed using scikit-learn's built-in `compute_class_weights` function with the balanced setting. This assigns each class a weight inversely proportional to its frequency. Below is a table with each class and its respective assigned weight:

Class	Weight
BENIGN	0.1038
DoS Hulk	1.0208
PortScan	1.4842
DDoS	1.8425
DoS GoldenEye	22.9186

FTP-Patator	29.7184
SSH-Patator	39.9982
DoS slowloris	40.6969
DoS Slowhttptest	42.8988
Bot	119.9693
Web Attack - Brute Force	156.4773
Web Attack - XSS	361.5166

These weights were passed through to all three classifiers during their training. The models were penalized more heavily for misclassifying the rare classes than the common classes. For example, a wrong Web Attack - XSS prediction is registered as 361.5 times more costly than a wrong BENIGN prediction.

Preprocessing

Using stratified sampling, the dataset was split into training (80%) and testing (20%) sets. This preserves the original class proportions in both partitions. All features were standardized using StandardScaler fitted exclusively on the training set and applied to the test set, preventing data leakage.

Models

Three supervised classifiers were trained with class weights applied to all:

- **Random Forest:** An ensemble of 100 decision trees with bootstrap sampling. Selected as the primary model for its robustness to noise, resistance to overfitting, and native feature importance scoring.
- **Decision Tree:** A single tree classifier is included to directly demonstrate the performance advantage of ensemble averaging over its base learner.
- **Logistic Regression:** A linear classifier included as a baseline to evaluate whether a linear decision boundary is sufficient for this multi-class problem.

All models were evaluated using macro-averaged precision, recall, and F1-score. Macro averaging treats all classes equally, regardless of sample size, providing an honest assessment of performance on rare attack classes that would otherwise be masked by weighted averages dominated by BENIGN traffic.

Results

Overall Model Performance

The table below summarizes the performance across all three models using macro-averaged metrics:

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	F1 (weighted)
Random Forest	0.9987	0.9129	0.9417	0.9235	0.9988
Decision Tree	0.9986	0.9190	0.9127	0.9152	0.9986
Logistic Regression	0.8342	0.3964	0.9177	0.4663	0.8820

Random Forest achieved the highest macro F1-score of 0.9235 by slightly outperforming Decision Tree with a macro F1-score of 0.9152 and significantly outperforming Logistic Regression with a macro F1-score of 0.4663. The weighted F1-score for Logistic Regression appears on par with the other two models' weighted F1-score because of the dominance of BENIGN traffic. The proper metric to look at is the macro F1-score, as it reflects the performance across all classes equally.

Per-Class Performance – Random Forest

The Random Forest classifier obtained a near-perfect performance across most attack classes. Below is a table of the per-class metrics:

Class	Precision	Recall	F1-score	Support
BENIGN	1.00	1.00	1.00	454,619
Bot	0.71	0.97	0.82	393
DDoS	1.00	1.00	1.00	25,606
DoS GoldenEye	1.00	1.00	1.00	2,059
DoS Hulk	1.00	1.00	1.00	46,215
DoS Slowhttptest	1.00	0.99	0.99	1,100
DoS slowloris	0.99	0.99	0.99	1,159

FTP-Patator	1.00	1.00	1.00	1,588
PortScan	0.99	1.00	1.00	31,786
SSH-Patator	1.00	1.00	1.00	1,179
Web Attack - Brute Force	0.84	0.71	0.77	301
Web Attack - XSS	0.43	0.65	0.52	130

Analysis of Difficult Classes

The three classes that had lower performance with Random Forest are as follows:

- Web Attack - XSS (F1: 0.52): The most challenging class across all models. XSS and Brute Force attacks share similar HTTP traffic patterns at the network flow level. Both involve repeated requests to web ports with similar packet size profiles, which makes them difficult to distinguish without application-layer payload inspection, which flow-level features do not capture.
- Web Attack - Brute Force (F1: 0.77): Moderate performance. 85 of 301 Brute Force samples were misclassified as XSS, reflecting the same inter-class similarity described above. Recall of 0.71 indicates the model misses approximately 29% of real Brute Force attacks.
- Bot (F1: 0.82): High recall (0.97) but lower precision (0.71) indicates the model over-predicts Bot traffic, generating false alarms from other classes being misclassified as Bot. This is partly a consequence of the high class weight (119.97) assigned to counteract Bot's rarity.

These results highlight an important practical limitation: network flow features alone are insufficient to perfectly distinguish all attack types, particularly those operating at the application layer. This is expected behavior and not a failure of the modeling approach. It reflects the inherent information boundary of flow-level telemetry.

Feature Importance – Random Forest

Below is a table with the top five features in classifying attack network traffic:

Rank	Feature	Importance
1	Destination Port	0.0953
2	Init_Win_bytes_backward	0.0743
3	Init_Win_bytes_forward	0.0557
4	Flow IAT Mean	0.0490
5	Bwd Packet Length Max	0.0480

Notably, Destination Port dominated the importance rankings. The reasoning for this is that different attack types have strong, consistent associations with specific destination ports. From the results, we see that Random Forest is excellent in exploiting this discrete signal. FTP-Patator consistently targets port 21, SSH-Patator targets port 22, and web-based attacks target ports 80 and 443. This makes Destination Port the single most efficient first split for the model. It immediately narrows the prediction space for several attack classes before any other feature is considered.

Logistic Regression Analysis

Logistic Regression performed poorly on this problem despite its high recall across most classes. Its macro precision of 0.3964 indicates that the model had extreme over-prediction of rare classes. For example, Bot precision of 0.01 means 99% of its Bot predictions are false alarms. This behavior is a direct consequence of class weights pushing a linear model beyond its capacity: the linear decision boundaries cannot cleanly separate 12 classes in 47-dimensional space, causing the model to cast overly wide prediction regions for heavily weighted rare classes.

The Logistic Regression coefficients do, however, provide interpretable insight into which features are most diagnostic for each attack type. For example, high Destination Port coefficients for FTP-Patator and SSH-Patator confirm the model learned to associate specific port numbers with specific brute-force attacks, consistent with the feature importance findings.

Limitations and Future Work

There are a few limitations that should be considered when interpreting these results:

- Lab-generated data: CICIDS2017 was produced in a controlled environment. Real-world network traffic is considerably noisier, attack patterns may deviate from the clean signatures captured in the dataset, and novel attack variants may not resemble training examples. Production performance would likely be lower than reported here.
- Flow-level feature ceiling: Web-based attacks (XSS, Brute Force) are inherently difficult to classify using network flow features alone, as their distinguishing characteristics exist at the HTTP payload level. Integrating deep packet inspection or application-layer features would be required for reliable web attack differentiation.
- Static model: The current model is trained once on historical data. Attack techniques evolve over time, and a deployed system would require periodic retraining or online learning to maintain effectiveness.
- No real-time pipeline: The system operates in batch mode on pre-computed flow features. Operational deployment would require integration with a live flow exporter such as NetFlow or a SIEM platform.
- Removed rare classes: Heartbleed, Infiltration, and SQL Injection were excluded due to insufficient samples. A production system should incorporate dedicated detection mechanisms for these high-severity threats despite their rarity.

Future work could address these limitations by incorporating additional data sources, implementing continuous model retraining, exploring deep learning approaches for packet-level feature extraction, and extending the pipeline to real-time network monitoring.

Conclusion

This project demonstrated that a Random Forest classifier with class weighting can reliably detect and classify 12 categories of network traffic, achieving a macro F1-score of 0.9235 across highly imbalanced classes on the CICIDS2017 benchmark. The system correctly identified nearly all instances of DDoS, DoS variants, FTP-Patator, SSH-Patator, and PortScan attacks, while showing expected difficulty with web application attacks that share similar network-layer signatures.

Random Forest is recommended for production deployment over Decision Tree and Logistic Regression. Its ensemble architecture provides robustness to noise and unseen attack variants that a single Decision Tree cannot guarantee, while Logistic Regression's linear decision boundaries proved fundamentally insufficient for this 12-class problem. The accompanying interactive Streamlit dashboard provides security analysts with accessible tools for exploring threat distributions, evaluating model performance, and investigating individual flow classifications without requiring direct code access.

The interactive dashboard accompanying this report is publicly accessible at:

<https://multiclassnidscicids2017.streamlit.app>

Course Connections

The following table maps project components to courses in my MS Data Science program:

Course	Application in This Project
Machine Learning	Random Forest, Decision Tree, Logistic Regression; class weighting; macro vs weighted evaluation metrics
Statistics for Data Science	Class imbalance analysis; correlation-based feature selection; distributional analysis of network features
Business Intelligence	Streamlit dashboard with interactive filters, log-scale visualizations, and per-class performance charts
Multivariate Analysis	Correlation heatmap; multicollinearity removal; Logistic Regression coefficient interpretation across 12 classes
Scripting Languages	End-to-end Python pipeline: multi-file ingestion, encoding repair, feature engineering, model training, export
Big Data Security	Network flow feature interpretation; CICIDS2017 attack taxonomy; threat classification framework